



University of Colombo, Sri Lanka



UCSC *University of Colombo School of Computing*

BACHELOR OF SCIENCE IN INFORMATION SYSTEMS

Second Year Examination — Semester I– UCSC AY22 [held in Sep/Oct 2025]

IS2203 — Object Oriented Programming

(Two (2) Hours)

Answer ALL questions



62

Number of Pages = 16

Number of Questions = 3

To be completed by the candidate

Index Number

--	--	--	--	--	--	--	--

Important Instructions to candidates:

- Students should answer in the medium of English language only using the space provided in this question paper.
- Note that questions appear on both sides of the paper. If a page is not printed, please inform the supervisor immediately.
- Write your index number **CLEARLY** on each and every page of this question paper.
- The duration of the paper is **Two (2)** hours.
- This paper has **3** questions in **16** pages (including the Cover Page).
- Answer **all** the questions.
- Question **1** carries **40 marks** and Question **2** and **3** carry **30 marks** each.
- Calculators and any electronic device capable of storing and retrieving text including electronic dictionaries, smart watches and mobile phones are not allowed.
- Do not tear off any part of this question paper. Under no circumstances may this paper, used or unused, be removed from the Examination Hall by a candidate.

To be completed by the examiners

1	
2	
3	

Index Number

--	--	--	--	--	--	--	--

1. Only ONE answer is correct in the following 20 MCQs.

Cross or color the **best-suited answer** among the given options.

Note: Assume that all the other required lines of codes are in place to run all given programs.

[2 x 20 = 40 marks]

(a).	(i)	(ii)	(iii)	(iv)	(v)
(b).	(i)	(ii)	(iii)	(iv)	(v)
(c).	(i)	(ii)	(iii)	(iv)	(v)
(d).	(i)	(ii)	(iii)	(iv)	(v)
(e).	(i)	(ii)	(iii)	(iv)	(v)
(f).	(i)	(ii)	(iii)	(iv)	(v)
(g).	(i)	(ii)	(iii)	(iv)	(v)
(h).	(i)	(ii)	(iii)	(iv)	(v)
(i).	(i)	(ii)	(iii)	(iv)	(v)
(j).	(i)	(ii)	(iii)	(iv)	(v)

(k).	(i)	(ii)	(iii)	(iv)	(v)
(l).	(i)	(ii)	(iii)	(iv)	(v)
(m).	(i)	(ii)	(iii)	(iv)	(v)
(n).	(i)	(ii)	(iii)	(iv)	(v)
(o).	(i)	(ii)	(iii)	(iv)	(v)
(p).	(i)	(ii)	(iii)	(iv)	(v)
(q).	(i)	(ii)	(iii)	(iv)	(v)
(r).	(i)	(ii)	(iii)	(iv)	(v)
(s).	(i)	(ii)	(iii)	(iv)	(v)
(t).	(i)	(ii)	(iii)	(iv)	(v)

(a). Which of the following statement best explains the difference between Object-Oriented Programming (OOP) and Procedural Programming?

- i. OOP breaks problems into small code steps, while Procedural Programming breaks them into objects.
- ii. Procedural Programming hides data inside classes, while OOP only runs step-by-step functions.
- iii. OOP combines data and functions inside objects, while Procedural Programming keeps data and functions separate.
- iv. Both OOP and Procedural Programming treat data and actions as always together.
- v. Procedural Programming supports inheritance and polymorphism, while OOP does not.

Index Number

--	--	--	--	--	--	--	--

(b). Consider the following statements about access specifiers.

Assertion (A): Private members of a class cannot be accessed directly from outside the class.

Reason (R): Private members are accessible only to member functions and friend functions of that class.

- i. Both (A) and (R) are true, and (R) is the correct explanation of (A).
- ii. Both (A) and (R) are true, but (R) is not the correct explanation of (A).
- iii. (A) is true, but (R) is false.
- iv. (A) is false, but (R) is true.
- v. Both (A) and (R) are false.

Consider the following code written in C++ and answer the questions from c to f.

```
class Temperature {
    int celsius;
public:
    Temperature (float c) {celsius = c;}
    operator float() { return (celsius * 9.0/5.0) + 32;}
    void show() {
        cout << "Temperature in Celsius: " << celsius << endl;
    }
};

int main() {
    Temperature t1 = 36.5;
    t1.show();
    float temp = t1;
    cout << "Assigned to float: " << temp << endl;
    return 0;
}
```

(c). When the statement `Temperature t1 = 36.5;` is executed, which value is actually stored inside the object?

- | | | |
|-------------------|----------------------|------------------------|
| i. 36 (Celsius) | ii. 36.5 (Celsius) | iii. 97.7 (Fahrenheit) |
| iv. Garbage value | v. Compilation error | |

Index Number

--	--	--	--	--	--	--	--

- (d). Which of the following statements correctly describes the behavior of the operator `float()` function in the `Temperature` class?

- i. It allows implicit conversion of `Temperature` objects to `float`.
- ii. It returns Celsius values directly.
- iii. It must always return an integer value.
- iv. It can take one parameter to customize the conversion.
- v. It is executed only when explicitly called, not during assignment.

- (e). What will be the output of the program?

- i. Temperature in Celsius: 36.5
Assigned to float: 36.5
- ii. Temperature in Celsius: 36.5
Assigned to float: 97.7
- iii. Temperature in Celsius: 36
Assigned to float: 96.8
- iv. Temperature in Celsius: 0
Assigned to float: 0
- v. Compilation error

- (f). What happens if one replaces the statement

```
cout << "Assigned to float: " << temp << endl;
```

with the statement

```
cout << t1; ?
```

- i. It prints the Celsius value.
- ii. It prints the Fahrenheit value.
- iii. A compilation error occurs.
- iv. It prints a garbage value.
- v. It prints nothing.

--	--	--	--	--	--	--	--

(g). What is the output of the following program written in C++?

```
namespace IS2102 {
    int add(int a, int b) { return a + b; }
}
namespace IS2103 {
    int add(int a, int b) { return a * b; }
}

int main() {
    using namespace IS2102;
    cout << IS2102::add(2, 3) << endl;
    cout << IS2103::add(2, 3) << endl;
    cout << add(4, 5) << endl;
}
```

i. 5 6 9

ii. 6 5 20

iii. 5 6 20

iv. 0 0 0

v. Compilation error

Consider the following code written in C++ and answer the questions from h to i.

```
class Counter {
    static int count;
public:
    Counter() { count++; }
    static void show() { cout << count << " "; }
};
int Counter::count = 0;

int main() {
    Counter c1, c2;
    c1.show();
    Counter::show();
}
```

(h). Which of the following statements about this program is true?

- i. The static variable count belongs to each object separately.
- ii. Static member functions can only be accessed through objects, not using the class name.
- iii. Both c1.show() and Counter::show() call the same function and print the same value.
- iv. The constructor cannot modify a static variable.
- v. Static variables are destroyed when an object goes out of scope.

Index Number

--	--	--	--	--	--	--	--

(i). What will be the output of the program?

- | | | |
|---------|----------------------|----------|
| i. 1 1 | ii. 2 2 | iii. 2 0 |
| iv. 0 2 | v. Compilation error | |

(j). What is the output of the following program written in C++?

```
class Student {
public:
    int marks;
    Student(int m) { marks = m; }
    void display() { cout << marks << endl; }
};

int main() {

    Student s1(90);
    Student* ptr = &s1;
    ptr->marks=85;

    cout << s1.marks << ":";
    cout << ptr->marks << ":";
    ptr->display();
}
```

- | | | |
|--------------|----------------------|---------------|
| i. 90:90:90 | ii. 85:90:85 | iii. 90:85:85 |
| iv. 85:85:85 | v. Compilation error | |

(k). Consider the following three statements regarding the Inheritance in OOP.

- I. It allows to have all child class attributes in the parent class.
- II. It is a mechanism that allows to use method overriding.
- III. Train and Engine have the relationship define as Inheritance.

Which of the above statement(s) is/are TRUE?

- | | | |
|---------------|--------------------|-----------------------|
| i. I only. | ii. II only. | iii. II and III only. |
| iv. III only. | v. I, II, and III. | |

Index Number

--	--	--	--	--	--	--	--

(l). Which of the following is TRUE according to the following class definition?

```
class C : private A, protected B {  
    // body of the class  
}
```

- i. All public, protected and private data in A will become private to C.
- ii. Both public and protected data in B will become protected to C.
- iii. Both private and protected data in B will become protected to C.
- iv. only public data in A will become private to C.
- v. C can access any data in B.

(m). What is the output of the following program written in C++?

```
class A{  
    public:  
        A() {cout<<"A";}   
        A(int x) {cout<<x;}};  
  
class B: private A {  
    public:  
        B() {cout<<"B";}   
        B(int x) {cout<<x;}};  
  
int main(){ B(50); }
```

- | | | |
|------------|---------|-----------|
| i. 50A | ii. A50 | iii. 50AB |
| iv. A50B50 | v. 50 | |

(n). Consider the following three statements regarding the Diamond Problem in OOP.

- I. It is a runtime error.
- II. It can cause ambiguity when accessing members of the common base.
- III. It can be solved using function overloading.

Which of the above statement(s) is/are TRUE?

- | | | |
|---------------|--------------------|-----------------------|
| i. I only. | ii. II only. | iii. II and III only. |
| iv. III only. | v. I, II, and III. | |

Index Number

--	--	--	--	--	--	--	--

- (o). Consider the following three statements regarding the Virtual Functions in C++.

- I. They make a class abstract.
- II. They are resolved at run time.
- III. They cannot have a body in the base class.

Which of the above statement(s) is/are TRUE?

- | | | |
|---------------|--------------------|-----------------------|
| i. I only. | ii. II only. | iii. II and III only. |
| iv. III only. | v. I, II, and III. | |

- (p). What is the output of the following program written in C++?

```
class ABC {
public:
    int func(char x, int y) {cout<<"X";}
    int func(int x, char y) {cout<<"Y";}
    int func(char y) {cout<<y;}
};

int main() {
    ABC obj;
    obj.func(65);
    obj.func(65, 'A');
}
```

- | | | |
|--------|----------------------|-----------|
| i. XY | ii. AY | iii. A65A |
| iv. yY | v. Compilation Error | |

- (q). Consider the following three statements regarding the function definition given below.

```
virtual int f(int, int) = 0;
```

- I. This function returns 0 when it calls.
- II. You must override this function before use.
- III. You cannot create objects of the class that contains this function.

Which of the above statement(s) is/are TRUE?

- | | | |
|----------------------|--------------------|---------------|
| i. I only. | ii. I and II only. | iii. II only. |
| iv. II and III only. | v. I, II, and III. | |

Index Number

--	--	--	--	--	--	--	--

(r). Consider the following three statements regarding the Encapsulation in OOP.

- I. It prevents a class from having any public members.
- II. It can only be achieved using abstract classes.
- III. It helps in data hiding by using access specifiers.

Which of the above statement(s) is/are TRUE?

- | | | |
|---------------|--------------------|---------------|
| i. I only. | ii. I and II only. | iii. II only. |
| iv. III only. | v. I, II, and III. | |

(s). Consider the following three statements regarding the templates in C++.

- I. They are resolved at runtime.
- II. They help in reducing code duplication.
- III. The compiler generates separate versions of a template for each type used.

Which of the above statement(s) is/are TRUE?

- | | | |
|----------------------|--------------------|---------------------|
| i. I only. | ii. II only. | iii. I and II only. |
| iv. II and III only. | v. I, II, and III. | |

(t). Consider the following three statements regarding exceptions in OOP.

- I. They are resolved at compile time.
- II. An exception must always be of type `int`.
- III. More than one exception can be caught in a single program.

Which of the above statement(s) is/are TRUE?

- | | | |
|---------------|--------------------|----------------------|
| i. I only. | ii. II only. | iii. I and III only. |
| iv. III only. | v. I, II, and III. | |

--	--	--	--	--	--	--	--

- accountNumber (integer)
- accountHolder (string)
- balance (float)

```
class BankAccount {
    private:
        int accountNumber;
        string accountHolder;
        float balance;

    public:
        // (a).i Default constructor
        // (a).ii Parameterized constructor
        // (a).iii Copy constructor
        // (a).iv Destructor
        // (b) Display function
        // (c) Overload + operator
        // (d) Friend function signature
};

int main() {
    // (e)
    return 0;
}
```

- i. A default constructor that sets
 - `accountNumber = 0`
 - `accountHolder = "Unknown"`
 - `balance = 0.0`using a constructor initialization list.

10

Index Number

--	--	--	--	--	--	--	--

- ii. A parameterized constructor to initialize all attributes using a constructor initialization list.

--

[2.5 marks]

- iii. A copy constructor that creates a new object as a copy of an existing BankAccount.

--

[2.5 marks]

- iv. A destructor that prints a message indicating the account is being deleted.

--

[2.5 marks]

--	--	--	--	--	--	--	--

1. The first step in the process of identifying a problem is to recognize that a problem exists. This is often done by comparing current performance with a desired state or goal. If there is a significant difference, a problem is identified. For example, if a company's sales are declining while its competitors' are increasing, this indicates a problem that needs to be addressed.

[3 marks]

- The balances are summed,
- The account holder names are concatenated with an ‘&’ separator, and
- A new account number is assigned to the resulting joint account.

[5 marks]

Index Number

--	--	--	--	--	--	--	--

- (d). Define a friend function named `compareBalance` that takes two `BankAccount` objects as arguments. The function should return the account with the higher balance. If both accounts have the same balance, it can return either one.

--

[4 marks]

- (e). Within the `main()` function define the following.
- Create three `BankAccount` objects: `acc1`, `acc2`, and `acc3` respectively using the default constructor, parameterized constructor, and the copy constructor.
 - Create a joint account object called `jointAcc` using the overloaded `+` operator.
 - Display the details of `acc1`, `acc2`, `acc3`, and `jointAcc` objects.

--

[8 marks]

Index Number

--	--	--	--	--	--	--	--

3. (a). Write the output of each of the following C++ programs.
If a program resulting a compilation error, write "Compiler Error" as the answer.
Note that all syntax rules (such as proper use of semicolons and balanced brackets) are correct in these programs.

```
i. class A {
    public: A() {cout<<"A";} };

class B {
    public: B() {cout<<"B";}
           B(int y) {cout<<"yB";} };

class C: public A, public B {
    public: C() {cout<<"C";}
           C(int y) {cout<<"yC";} };

int main() {
    C c(1);
}
```

[3 marks]

```
ii. class A {
    public:
        A() {cout<<"A";}
        int f1(int x) {return x+2;} };

class B: private A {
    public:
        B() {cout<<"B";}
        int f2(int x) {return f1(x+1);} };

int main( ) {
    B b;
    cout<<b.f2(3); }
```

[3 mark]

```
iii. class A {
    public:
        void f() {cout<<"void";}
        int f() {cout<<"int";}
        char f() {cout<<"char";}

int main() { A a; a.f(); }
```

[3 mark]

Index Number

--	--	--	--	--	--	--	--

iv. `template <typename T, typename U, int n=2>
T f(T x, U y) { return x + y * n; }`

```
int main() {  
    cout << f<char, int, 3>('A', 1);  
    cout << f<int, char>('B', 3);  
}
```

[3 mark]

.....

v. `template <typename T, typename U>
class A {
 public:
 T f(U x) { return x+1; }
};`

```
int main() {  
    A<char, int> a;  
    cout << a.f(65);  
}
```

[3 mark]

.....

vi. `int main() {
 try { throw 'a'; cout << "A"; }
 catch (int x) { cout << "B"; }
 catch (char x) { cout << "C"; }
 catch (...) { cout << "D"; }
 cout << "E";
}`

[3 mark]

.....

vii. `int main() {
 try { cout << "A"; throw 'a'; }
 catch (char x) {
 try { cout << "B"; throw 'b'; }
 catch (int x) { cout << "C"; }
 catch (...) { cout << "D"; }
 }
 cout << "E";
}`

[3 mark]

.....

Index Number

--	--	--	--	--	--	--	--

(b). Use the following class definition to answer the questions below.

```
class Customer {
    string name;
public:
    Customer(string n) { name = n; }
    string getName() { return name; } };

class BankAccount {
    Customer owner; int balance;
public:
    BankAccount(string n, int x) : owner(n), balance(x) {}
    void deposit(int x) { balance = balance + x; }
    void withdraw(int x) { balance = balance - x; }
    int checkBalance() { return balance; }
    void printName() { ---(A)--- } };
```

i. What is the relationship between Customer and BankAccount objects?

--

[2 marks]

ii. Implement the printName function mentioned as (A) in the above code snippet to print the owner name when it is called.

--

[3 marks]

iii. Write the main() function that performs the following operations:
Creates a BankAccount object for your name with an initial balance of 100, deposits 250 and withdraws 100. Prints the account holder's name using the appropriate function, and finally, prints the remaining balance.

--

[4 marks]
